MCTR911 – Robotics Programming

Milestone 3 Report

Velocity and Acceleration Kinematics of the UR5e, their Validation in MuJoCo, and the Cell Environment

Team 5

Name	ID
Mahmoud Ghobashy	58-2432
Kevin Ezzat	58-0408
Seif Hassan	58-6034
Mohamed Elsayed	58-16392
Fawzy Karim	58-9263

German University in Cairo – Mechatronics Department – Winter 2026

github.com/Mabrouukk/MCTR911-Robotics-Programming

Contents

1. Introduction.....
2. Starting point: DH table and forward kinematics.....
3. Velocity kinematics.....
  3.1 The geometric Jacobian.....
  3.2 Two derivations that must agree.....
  3.3 Forward velocity kinematics.....
  3.4 Inverse velocity kinematics.....
  3.5 Singularities.....
4. Acceleration kinematics.....
  4.1 From velocity to acceleration.....
  4.2 The derivative of the Jacobian, J̇.....
  4.3 Forward and inverse acceleration kinematics.....
5. Validation in the MuJoCo simulator.....
  5.1 What the simulator measures.....
  5.2 Test 1: joint values put straight into the simulator.....
  5.3 Test 2: while the physics runs.....
  5.4 Test 3: inverse kinematics driving the robot.....
  5.5 Test 4: near a singularity.....
  5.6 The same tests through ROS 2.....
6. The GUI and the cell environment.....
7. Videos.....
8. How to run.....
9. Conclusion.....
References.....

1. Introduction

In Milestone 2 we built the DH table of the UR5e, wrote its forward and inverse position kinematics, linked ROS 2 to MuJoCo, and ran the colour sorting cell. In this milestone we:

- derived the forward and inverse velocity kinematics of the arm (the geometric Jacobian J and its inverse),
- derived the forward and inverse acceleration kinematics (which needs the time derivative of the Jacobian, $\dot{J}$),
- tested all of them, together with the position kinematics, on the MuJoCo simulator: we put joint angles, rates and accelerations into the simulator, read its end-effector sensors, and compared with our equations; then we did the reverse, from the sensed end-effector values back to the joint values,
- used the inverse kinematics to drive the robot along a path in task space and measured how closely it follows it,
- studied the singularities of the arm and how our inverse handles them, and
- kept building the GUI: the cell now has a safety fence, people, chairs, an operator desk, a workbench with tools, a second conveyor, shelving and more, plus a live kinematics panel in the simulator window.

All the code is in the `Milestone 03/code` folder of our repository. Every number and figure in this report comes from our own scripts (`derive_symbolic.py`, `validate_kinematics.py`, `kinematics_validator.py`), so anyone can run them again and get the same results. Section 8 lists the commands.

Throughout the report the end effector is the tool flange of the UR5e (the point where the gripper is mounted, called attachment_site in the MuJoCo model), the same point our Milestone 2 forward kinematics returns. All vectors are written in the robot base frame {0}.

2. Starting point: DH table and forward kinematics

We use the same standard DH convention and table as in Milestone 2:

Table 1: *UR5e DH parameters (standard convention), from Milestone 2.*

i	θi	di (m)	ai (m)	αi
1	q1	0.163	0	+90°
2	q2	0	−0.425	0
3	q3	0	−0.392	0
4	q4	0.134	0	+90°
5	q5	0.100	0	−90°
6	q6	0.100	0	0

Multiplying the six link transforms gives the flange pose:

```
T_0^6(q) = A1(q1) · A2(q2) · A3(q3) · A4(q4) · A5(q5) · A6(q6)
A_i = Rz(θi) · Tz(di) · Tx(ai) · Rx(αi)
```

The partial products $T_0^{(i-1)} = A1 \cdots A(i-1)$ are the frames of the joints. We need two things from each of them: its z axis $z(i-1)$ (third column of the rotation), which is the axis joint i turns about, and its origin $o(i-1)$ (the translation), a point on that axis. Our symbolic derivation (Section 3.2) prints the flange position in closed form:

```
px = d4 s1 + d5 s234 c1 + d6 (s1 c5 − s5 c1 c234) + (a2 c2 + a3 c23) c1
```

```
py = −d4 c1 + d5 s1 s234 − d6 (s1 s5 c234 + c1 c5) + (a2 c2 + a3 c23) s1
pz = d1 + a2 s2 + a3 s23 − d5 c234 − d6 s5 s234
(s1 = sin q1, c23 = cos(q2+q3), s234 = sin(q2+q3+q4), ...)
```

3. Velocity kinematics

3.1 The geometric Jacobian

Velocity kinematics answers: if the joints turn at rates $\dot{q} = (\dot{q}1, \ldots, \dot{q}6)$, how fast does the flange move? The answer is a linear map, the Jacobian:

$$\dot{x} = [\, v\ ;\ \omega\,] = J(q)\,\dot{q} \qquad J \text{ is } 6 \times 6$$

v is the linear velocity of the flange point (m/s) and ω its angular velocity (rad/s). We build J one column at a time. All six UR5e joints are revolute. Imagine only joint i moving, at rate $\dot{q}i$:

- The whole part of the arm after joint i turns about the axis z(i−1), so the flange gets the angular velocity $\omega = z(i-1)\,\dot{q}i$.
- The flange point p moves on a circle around that axis. A point turning about an axis through o has velocity $\omega \times (p - o)$, so $v = z(i-1) \times (p - o(i-1))\,\dot{q}i$.

Velocities add up when several joints move together, so column i of J is

$$J_i = [\, z(i-1) \times (p - o(i-1))\ ;\ \ z(i-1)\,]$$

In code (velocity_kinematics.jacobian) this is a loop over the six frames that our forward kinematics already returns: no new geometry is needed, only the FK frames.

The joint axes come out as (from the symbolic derivation):

```
z0 = (0, 0, 1)
z1 = z2 = z3 = (sin q1, −cos q1, 0)
z4 = (s234 c1, s1 s234, −c234)
z5 = (s1 c5 − s5 c1 c234, −s1 s5 c234 − c1 c5, −s5 s234)
```

Joints 2, 3 and 4 are parallel: their axes are all horizontal and perpendicular to the arm's plane. This is the typical UR layout, and it is the reason the arm has a closed-form inverse (Milestone 2) and simple singularities (Section 3.5).

3.2 Two derivations that must agree

In derive_symbolic.py we derive J symbolically with sympy, in two independent ways: with the formula above, and by differentiating the forward kinematics directly, $Jv = \partial p/\partial q$ (the linear rows). The two agree to $2 \cdot 10^{-16}$ at 50 random joint vectors, and the symbolic J agrees with our numerical function to $7 \cdot 10^{-16}$. Two columns are worth writing out:

```
J_1 = ( −py, px, 0, 0, 0, 1 )
J_6 = ( 0, 0, 0, z5 )
```

- Column 1: the base joint spins the flange about the vertical axis through the origin, so the flange moves sideways on a horizontal circle (no vertical part).
- Column 6: the last axis passes through the flange point itself, so turning joint 6 only rotates the flange and does not move it (linear part zero).

3.3 Forward velocity kinematics

Given q and $\dot{q}$, the flange twist is simply $\dot{x} = J(q)\,\dot{q}$. This is what the robot's own controller does to report the tool speed.

3.4 Inverse velocity kinematics

Given a desired flange twist $\dot{x}$ (for example, move at 5 cm/s along x without turning), find the joint rates. The UR5e has exactly six joints for a six-dimensional twist, so J is square and, away from singularities, invertible:

$$\dot{q} = J(q)^{-1}\,\dot{x}$$

In code we never form the inverse matrix; we solve the linear system $J\,\dot{q} = \dot{x}$ (numpy.linalg.solve), which is faster and more accurate.

3.5 Singularities

The inverse fails where det J = 0. For the UR5e the determinant has a short closed form, which we checked against the determinant of the symbolic J at 200 random joint vectors (difference $6 \cdot 10^{-17}$):

$$\det J = a2\ a3\ \sin(q3)\ \sin(q5)\ (\ a2\ \cos q2 + a3\ \cos(q2+q3) + d5\ \sin(q2+q3+q4)\ )$$

Each factor is one singularity:

Table 2: *The three singularities of the UR5e, the factors of det J.*

Singularity	Condition	What happens physically
Elbow	sin q3 = 0	The forearm and upper arm are in one line (arm fully stretched or folded flat). The flange cannot move along the arm.
Wrist	sin q5 = 0	The axes of joints 4 and 6 line up, so they do the same thing. One rotation of the flange is lost.
Shoulder	a2 c2 + a3 c23 + d5 s234 = 0	The wrist centre lies on the axis of joint 1, so turning the base cannot move it sideways.

Near a singularity the plain inverse asks for enormous joint rates: they grow like 1/σmin, where σmin is the smallest singular value of J. To stay safe we switch to damped least squares when σmin drops below 0.02 m/rad:

$$\dot{q} = J^{T}\,(J\,J^{T} + \lambda^{2}\,I)^{-1}\,\dot{x}, \qquad \lambda^{2} = \lambda\max^{2}\,(1 - (\sigma\min/\varepsilon)^{2}), \quad \varepsilon = 0.02,\ \lambda\max = 0.02$$

Away from the singularity λ = 0 and this is exactly the inverse. Near it, the motion along the lost direction is given up instead of asking for infinite speed. Figure 4 shows this: with the wrist at q5 = 0.05°, a flange speed of 5 cm/s needs 146 rad/s from the plain inverse (the UR5e limit is π rad/s), while the damped inverse asks for 0.02 rad/s. Our sorting motions stay far from all three singularities (σmin ≥ 0.23 m/rad).

4. Acceleration kinematics

4.1 From velocity to acceleration

Differentiating $\dot{x} = J(q)\,\dot{q}$ with respect to time (product rule) gives the acceleration of the flange:

$$\ddot{x} = [\ a\ ;\ \alpha\ ] = J(q)\,\ddot{q} + \dot{J}(q, \dot{q})\,\dot{q}$$

The first term is what the joint accelerations produce. The second term is there even when every joint turns at a constant speed: because the arm's shape changes, the Jacobian changes, and the flange is accelerated anyway (centripetal and Coriolis-like terms). It is not small: along our task-space test path (Section 5.4) it reaches 0.23 m/s², a third of the peak flange acceleration of 0.69 m/s².

4.2 The derivative of the Jacobian, $\dot{J}$

J depends on time only through q, so by the chain rule $\dot{J} = \Sigma k\ (\partial J/\partial qk)\ \dot{q}k$. Our symbolic script computes exactly that. For the code we use a faster recursive form. Differentiating column i with the product rule:

$$d/dt\ [\ z \times (p - o)\ ] = \dot{z} \times (p - o) + z \times (\dot{p} - \dot{o})$$
$$d/dt\ z = \dot{z} = \omega(i-1) \times z \qquad \text{(z is fixed in frame i−1, so it turns with that frame)}$$
$$\dot{p} = Jv\ \dot{q} \qquad \text{(flange velocity)}$$

The angular velocity ω and the origin velocity ȯ of each frame are built up from the base, link by link:

$$\omega_k = \omega_(k-1) + \dot{q}_k\ z_(k-1)$$
$$\dot{o}_k = \dot{o}_(k-1) + \omega_k \times (o_k - o_(k-1))$$

The recursive $\dot{J}$ (velocity_kinematics.jacobian_dot) agrees with the symbolic $\Sigma\ (\partial J/\partial qk)\ \dot{q}k$ to $2 \cdot 10^{-15}$ and with central finite differences of J to $5 \cdot 10^{-10}$.

4.3 Forward and inverse acceleration kinematics

$$\text{forward: } \ddot{x} = J\ \ddot{q} + \dot{J}\ \dot{q}$$
$$\text{inverse: } \ddot{q} = J^{-1}\ (\ \ddot{x} - \dot{J}\ \dot{q}\ )$$

The inverse uses the same solver as the inverse velocity, including the damping near singularities.

5. Validation in the MuJoCo simulator

5.1 What the simulator measures

MuJoCo computes positions, velocities and accelerations of bodies with its own algorithms (spatial-vector recursions), not with our DH table. We added six MuJoCo sensors to the flange site (ee_sensors.py): position, orientation, linear and angular velocity, linear and angular acceleration. They are the measurement we compare our equations with. Three details matter:

- MuJoCo's linear acceleration sensor works like a real accelerometer: it also feels gravity and reads a − g. We add g back to get the flange's real acceleration.
- The robot base sits 0.4 m above the floor on its pedestal, with no rotation. Positions are shifted by that offset; velocities and accelerations are not.
- After a physics step MuJoCo's accelerations and sensors still belong to the state before the step. We call mj_forward before reading, so the joint state and the flange readings are from the same instant.

For a pass we require the equations and the sensors to agree within a simple tolerance: 10^{-6} (m, rad, m/s, rad/s) and 10^{-5} for accelerations. Anything above floating-point round-off would mean a mistake in the equations.

5.2 Test 1: joint values put straight into the simulator

We set 1000 random joint states in the simulator (q in [−π, π], $\dot{q}$ in [−π, π] rad/s, which is the UR5e speed limit, $\ddot{q}$ in [−10, 10] rad/s²), let MuJoCo compute its sensors for exactly that state, and compared:

Table 3: *Test 1: 1000 random joint states. Every check passes; the errors are floating-point round-off.*

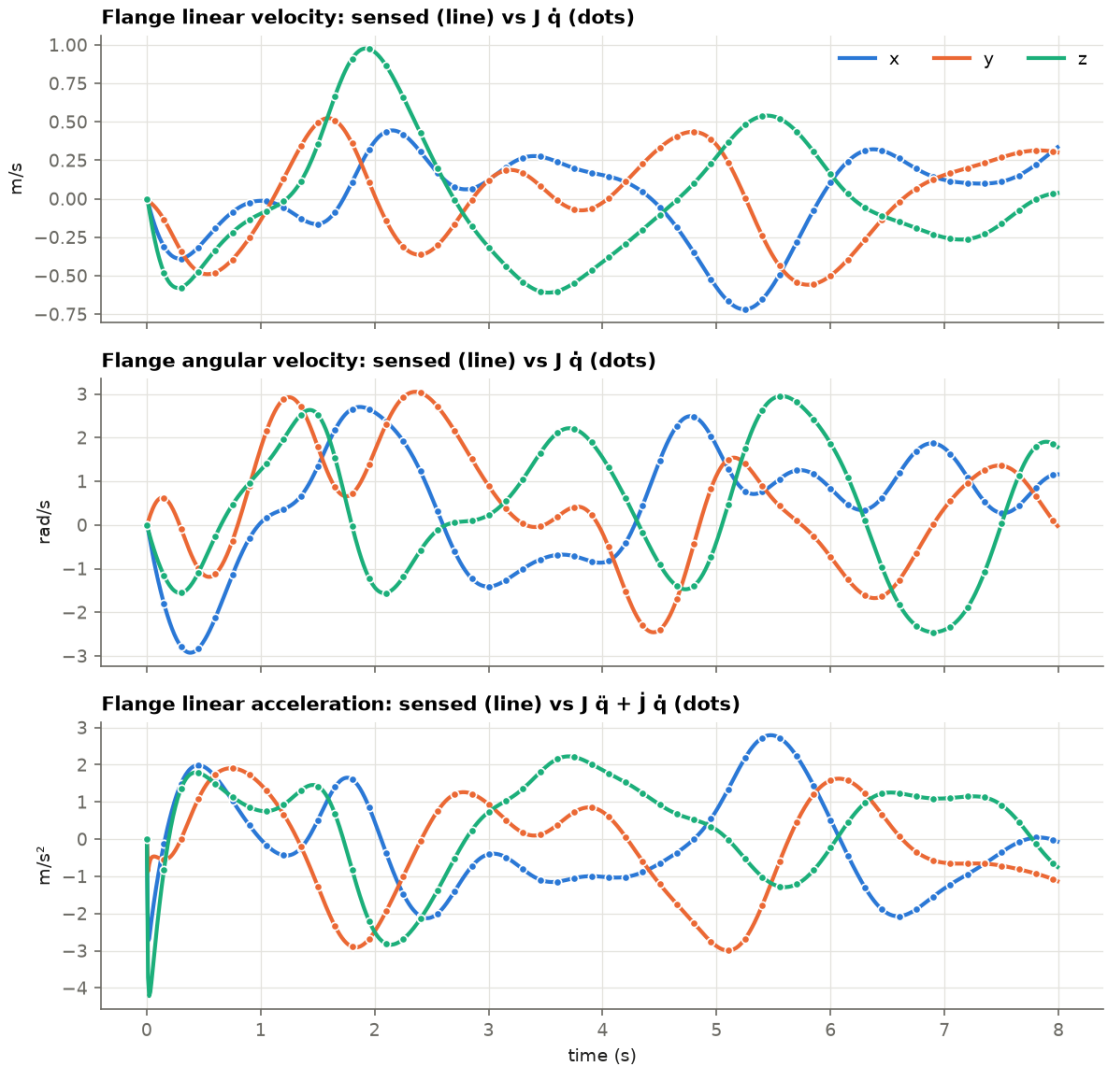
Check	Our equation	Compared with	Worst error
Forward position	FK(q)	sensed flange position	7e-16 m
Forward position	FK(q)	sensed flange orientation	1e-15 rad

Check	Our equation	Compared with	Worst error
Forward velocity	$J\,\dot{q}$	sensed linear velocity	4e-15 m/s
Forward velocity	$J\,\dot{q}$	sensed angular velocity	9e-15 rad/s
Forward acceleration	$J\,\ddot{q} + \dot{J}\,\dot{q}$	sensed linear acceleration	4e-14 m/s²
Forward acceleration	$J\,\ddot{q} + \dot{J}\,\dot{q}$	sensed angular acceleration	5e-14 rad/s²
Inverse position	IK(sensed pose)	the q we put in	8e-14 rad
Inverse velocity	J^{-1} (sensed twist)	the $\dot{q}$ we put in	5e-14 rad/s
Inverse acceleration	J^{-1} (sensed acc. − $\dot{J}\,\dot{q}$)	the $\ddot{q}$ we put in	3e-13 rad/s²

208 of the 1000 random states were close to a singularity ($\sigma\text{min} < 0.02$). There the inverse is not unique or is deliberately damped, so they are left out of the three inverse rows and treated separately in Test 4. The forward rows include all 1000 states.

5.3 Test 2: while the physics runs

Test 1 sets the state directly. In Test 2 the arm really moves: the position servos follow sine waves on all six joints (different amplitudes and frequencies), so the joint rates and accelerations change all the time. Every 10 ms we read the joint state and the flange sensors and repeated all nine checks. The flange reached 1.0 m/s and 5.0 m/s². Again every check passed, with worst errors of 6e-16 m, 6e-15 m/s and 5e-14 m/s² forward, and 5e-15 rad/s and 5e-14 rad/s² inverse.



Servos follow joint sine waves. Worst difference: velocity 2e-15 m/s, acceleration 1e-14 m/s² (floating-point round-off).

Figure 1: *Test 2, forward: the flange velocity and acceleration measured by the simulator (lines) and computed by our equations from the joint state (dots). The dots sit exactly on the lines.*

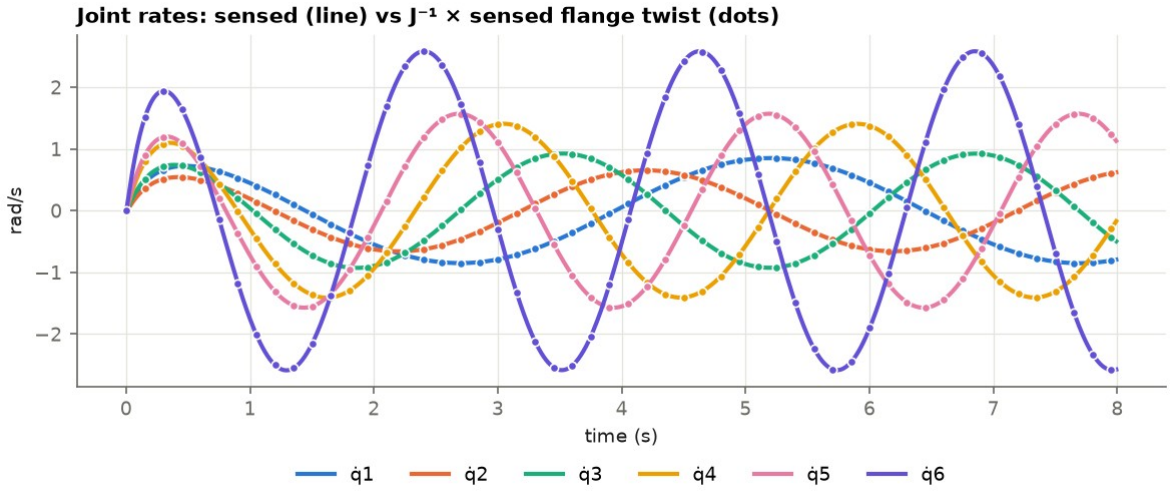


Figure 2: *Test 2, inverse: the joint rates measured by the simulator (lines) and computed from the sensed flange twist with J^{-1} (dots).*

5.4 Test 3: inverse kinematics driving the robot

In the tests so far the inverse equations only had to reproduce known joint values. In Test 3 they produce the motion. We planned a flange path in task space: a straight line from above the pick station to above the red bin, then a full horizontal circle of 8 cm radius, with the gripper pointing straight down the whole time and quintic timing (every segment starts and ends at rest). For every 2 ms of the path:

$$q = \mathrm{IK}(\, p(t), \text{gripper down}\,) \qquad \text{inverse position (nearest solution)}$$
$$\dot{q} = J^{-1}\,[\, v(t)\,;\,0\,] \qquad \text{inverse velocity}$$
$$\ddot{q} = J^{-1}\,(\,[\, a(t)\,;\,0\,] - \dot{J}\,\dot{q}\,) \qquad \text{inverse acceleration}$$

First, the three inverse results must agree with each other: the time derivative of the IK angles must equal $J^{-1}v$, and the derivative of that must equal the inverse acceleration. They do, to $5 \cdot 10^{-6}$ rad/s and $1 \cdot 10^{-3}$ rad/s² (the limit of numerical differentiation at 2 ms steps).

Then we sent the joint angles to the robot's position servos and measured where the flange really went:

Table 4: *Test 3: sensed flange path vs planned path (tolerance 5 mm).*

Run	Worst position error	Worst velocity error	Error at the end
1. Joint angles only	68.2 mm	114 mm/s	0.02 mm
2. Angles + velocity feed-forward	0.55 mm	2.2 mm/s	0.000 mm

Run 1 fails the tolerance, and the reason is the servo, not the kinematics. Each UR5e servo in the model is a spring and damper: torque = kp (q_cmd − q) − kd q̇. To move at speed q̇ it needs a position error of (kd/kp) q̇, and kd/kp = 0.2 s for every joint. So the arm trails the command by 0.2 s and cuts inside the circle. The fix uses our inverse velocity kinematics: we add that error to the command in advance,

$$q_{cmd} = q + (kd/kp) \cdot J^{-1}\,[\, v\,;\,0\,] \qquad \text{(velocity feed-forward)}$$

and the flange stays within 0.55 mm of the planned path (Figure 3). The gripper stays vertical within 0.03° in both runs.

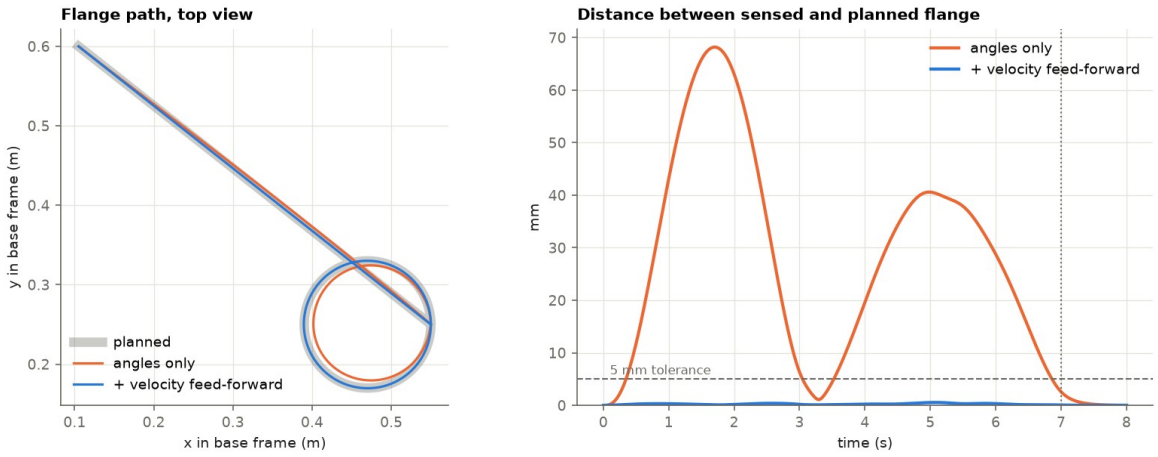


Figure 3: *Test 3: planned flange path (grey), and the path the flange really took with joint angles only (orange) and with velocity feed-forward from the inverse velocity kinematics (blue).*

5.5 Test 4: near a singularity

To see what the inverse does near a singularity we started from the home pose and brought the wrist towards q5 = 0 while asking for a flange speed of 5 cm/s along x. As q5 shrinks, σmin shrinks with it, and the plain inverse asks for joint rates that grow like 1/σmin, past the UR5e limit of π

rad/s at q5 = 2.3°. Once σmin < 0.02 m/rad the damped inverse takes over and the joint rates go down instead. The price is that the flange no longer gets exactly the requested speed along the lost direction, which is the right trade near a singularity. Away from it the damped inverse is exact (difference below 10^{-12}).

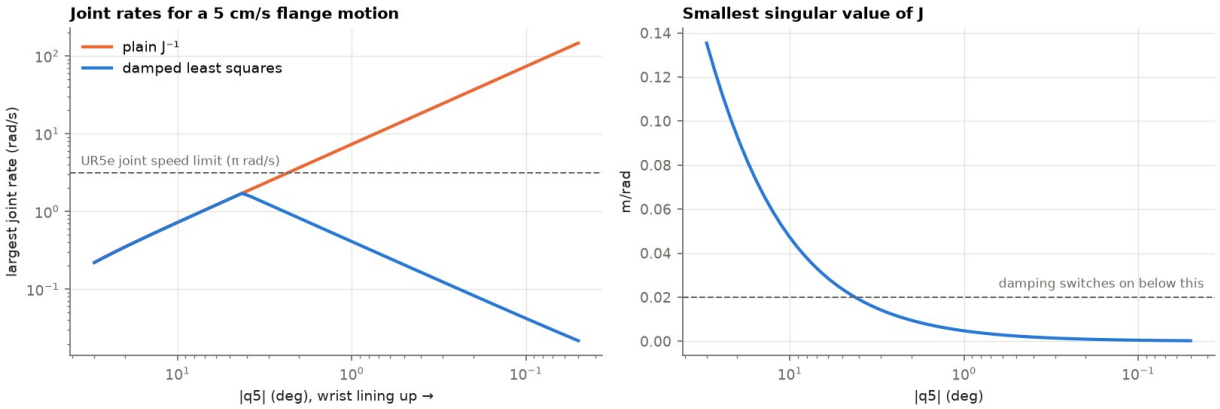


Figure 4: *Test 4: joint rates needed for a 5 cm/s flange motion as the wrist lines up (left), and the smallest singular value of J (right).*

5.6 The same tests through ROS 2

The tests above run without ROS 2, for speed and for the figures. The same checks also run live through our ROS 2 pipeline (`scripts/run_ms3.sh forward` and `inverse`). The simulator runs on the Mac. The bridge node in Docker publishes the joint states, joint accelerations and the sensed flange pose, twist and acceleration, all stamped with the simulator time of the same instant. Our `kinematics_validator` node drives the arm, matches the messages by their stamp and runs the nine checks on every instant:

Table 5: *Live validation through ROS 2.*

Run through ROS 2	Instants checked	Equations vs sensors	Flange vs planned path
forward (joint sine waves)	265	all 9 pass, worst $9 \cdot 10^{-14}$	–
inverse (task-space path, 50 Hz commands)	353	all 9 pass, worst $2 \cdot 10^{-14}$	0.70 mm (tolerance 5 mm)

Two practical lessons came from the ROS 2 run. First, commands are sent at 50 Hz and each one is held for 20 ms, which delays it by 10 ms on average; aiming each command at the middle of its hold window removes that delay. Second, the simulator time and the node's clock must be lined up carefully: using a state message that was up to 33 ms old as the start time showed up as up to 10 mm of false error.

6. The GUI and the cell environment

We continued building the environment of the colour sorting cell around the robot. Everything new is built from simple shapes in our environment file, placed outside the robot's working area and set not to collide, so the sorting task runs exactly as before (all six parts sorted in 83 s). The cell now contains:

Table 6: *Components of the cell environment.*

Group	Components
Safety	Guard fence with yellow posts and transparent panels, a gate (open), a light curtain on the infeed opening, a warning sign, emergency-stop buttons, a fire extinguisher

Group	Components
People	A seated operator at the HMI desk, a technician at the workbench, a worker walking along the fence with a clipboard (hard hats and safety vests)
Chairs	Operator chair, workbench chair, a stool
Control	Operator desk with an HMI screen (part counters, running lamp), keyboard and e-stop; control cabinet with a red / amber / green stack light
Tools	Workbench with a toolbox, wrench, screwdriver, hammer, spare finger pad, calibration block; tool board on the wall
Logistics	Infeed conveyor with a hopper and drive motor feeding the cell conveyor, a pallet with boxes, a shelving rack
From Milestone 2	UR5e with the Robotiq 2F-85 gripper on its pedestal, the cell conveyor, the colour sensor at the pick station, the three colour bins, floor markings

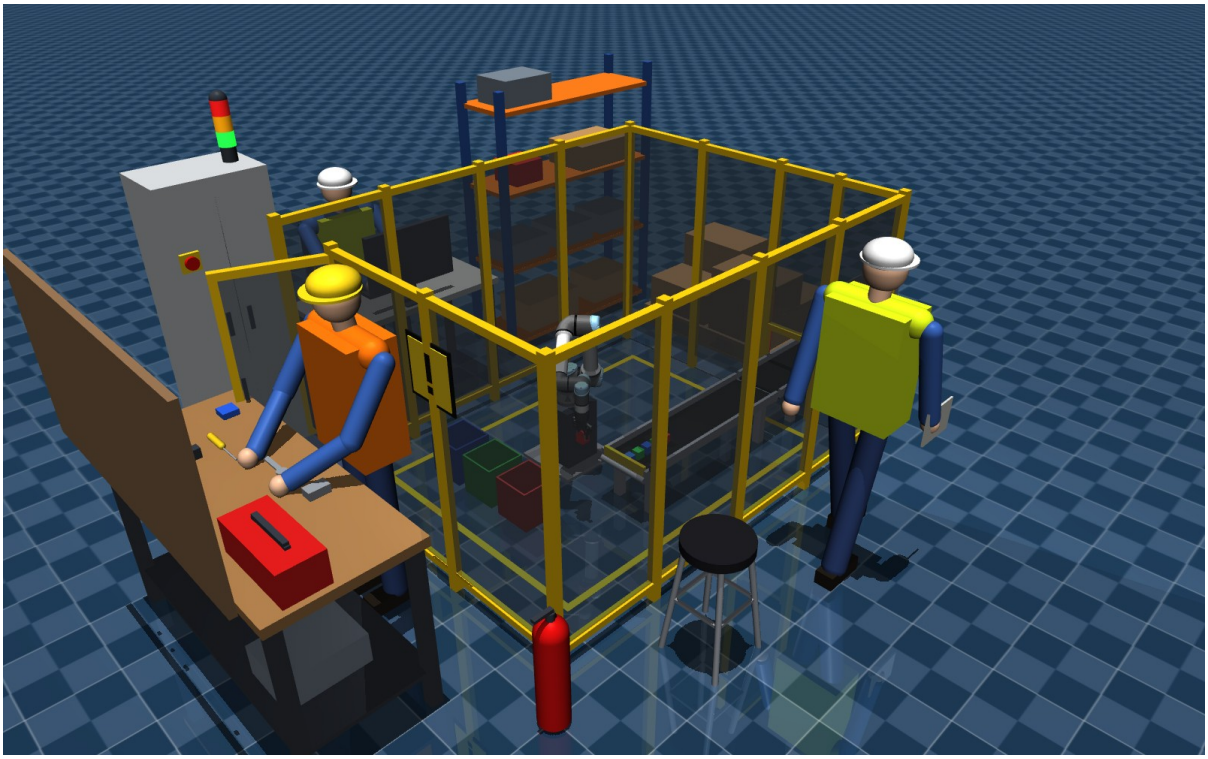


Figure 5: *The cell environment: the guarded UR5e cell with its conveyors and bins, the operator, technician and walking worker, workbench, control cabinet and shelving.*

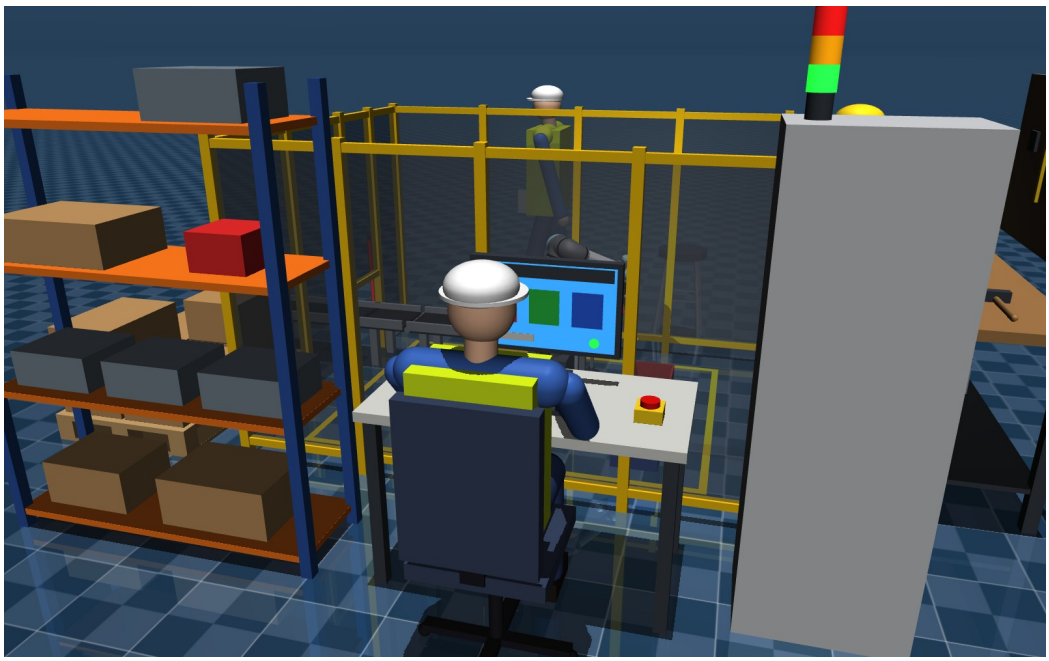


Figure 6: *Operator station: HMI desk with part counters and e-stop, control cabinet with stack light, shelving.*

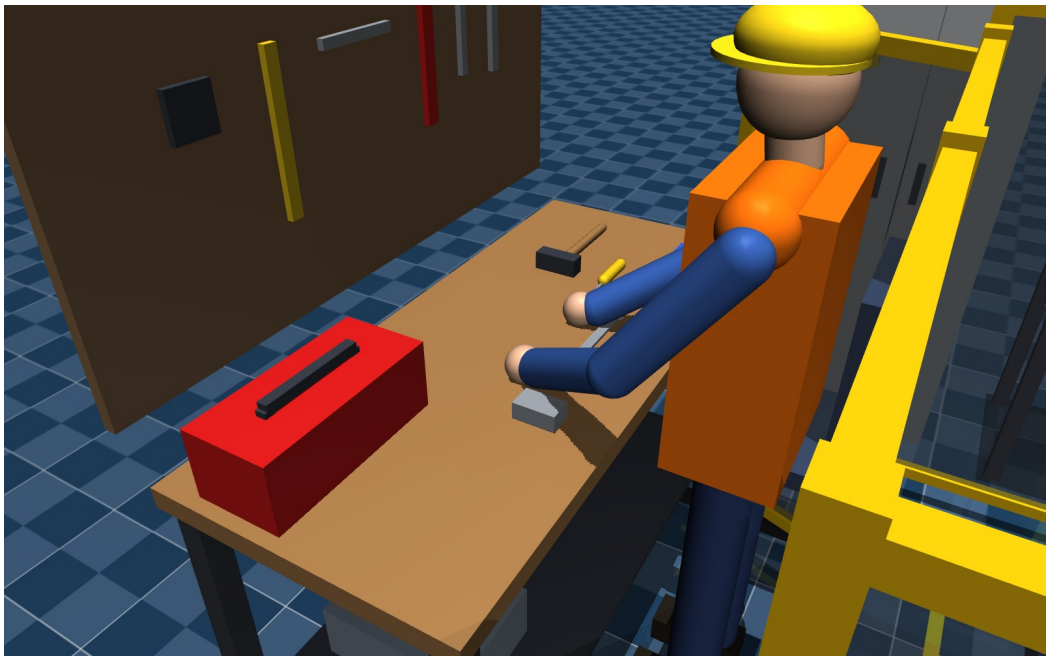


Figure 7: *Workbench with tools and the technician.*

The simulator window also has a new live panel (top right). While the arm moves, it compares our equations with the simulator's sensors at every update: one line each for position, velocity, acceleration and the inverse, showing OK or FAIL and the worst error so far, then the flange speed and the distance from a singularity. This makes the validation visible in the GUI itself, not only in a log.

The ROS 2 interface of the cell:

Table 7: *ROS 2 topics of the Milestone 3 bridge.*

Topic	Type	Content
/ur5e/joint_commands	Float64MultiArray	6 joint angles (+ gripper 0–255), input
/ur5e/joint_states	JointState	joint angles and rates
/ur5e/joint_accelerations	Float64MultiArray	simulator time + 6 joint accelerations

Topic	Type	Content
/ur5e/ee_pose	PoseStamped	sensed flange pose (base frame)
/ur5e/ee_twist	TwistStamped	sensed flange linear and angular velocity
/ur5e/ee_accel	AccelStamped	sensed flange linear and angular acceleration
/cell/station_color	String	colour of the part at the pick station

7. Videos

The videos are in Milestone 03/videos:

- ms3_kinematics_validation.mp4: the kinematics validation with comments on the results: joint angles in (forward and inverse position), joint rates and accelerations in (forward and inverse velocity and acceleration, with a live chart), the inverse kinematics driving the robot along the task-space path (angles only, then with velocity feed-forward), the wrist singularity, and a summary.
- ms3_gui_environment.mp4: a tour of the cell environment while the robot sorts parts.

8. How to run

```
scripts/run_ms3.sh forward     # simulator + ROS 2: joint-space test, live checks
scripts/run_ms3.sh inverse     # simulator + ROS 2: task-space path via IK / J⁻¹
scripts/run_ms3.sh sort        # colour sorting in the full environment
scripts/run_ms3.sh validate    # Tests 1–4 and the figures, no ROS 2 (6 s)
scripts/run_ms3.sh derive      # symbolic derivation with sympy (5 s)
```

9. Conclusion

We derived the forward and inverse velocity kinematics (the geometric Jacobian and its inverse) and the forward and inverse acceleration kinematics (with the analytical $\dot{J}$) of the UR5e, each in two independent ways that agree to round-off. In the MuJoCo simulator, the end-effector position, orientation, velocity and acceleration we compute from joint values match the simulator's sensors to better than 10^{-13}, and going back from the sensed end-effector values gives the joint values again: every check is far inside the tolerance, offline and live through ROS 2. Used to drive the robot, the inverse kinematics kept the flange within 0.6 mm of a task-space path, once the inverse velocity was also used to cancel the servo lag. We know where the three singularities are (det J in closed form) and the inverse stays bounded near them.

For Milestone 4 this gives us what trajectory planning and control need: the inverse velocity and acceleration kinematics to turn task-space trajectories into joint trajectories, and a first feed-forward term for the motor-level controller.

References

[1] M. W. Spong, S. Hutchinson and M. Vidyasagar, Robot Modeling and Control. Wiley, 2006.

[2] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, Robotics: Modelling, Planning and Control. Springer, 2009.

[3] Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," ASME Journal of Dynamic Systems, Measurement, and Control, vol. 108, no. 3, 1986.

[4] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," IEEE Transactions on Systems, Man, and Cybernetics, vol. 16, no. 1, 1986.

[5] E. Todorov, T. Erez and Y. Tassa, "MuJoCo: A physics engine for model-based control," IEEE/RSJ IROS, 2012.

[6] Universal Robots, "DH parameters for calculations of kinematics and dynamics," Universal Robots support article.